Dependencies & Resolution

1. How does Maven dependency resolution work, including transitive dependencies?

A: Maven resolves dependencies by reading your **pom.xml** and looking for all the libraries you listed. It then checks those libraries to see which *other* libraries they need (called **transitive dependencies**). Maven tries to download all of these required libraries from the repositories and adds them to your project's classpath automatically.

2. How do you handle or fix version conflicts between dependencies in Maven?

A: You fix version conflicts by using the **"nearest definition"** rule: the version defined closest to your project in the dependency tree wins. The simplest way to enforce a version is to explicitly list the correct version in the **<dependencyManagement>** section of your pom.xml. This forces Maven to use that single, consistent version everywhere, even if other libraries ask for an older version.

3. Scenario: You need a specific version of a dependency that conflicts with other libraries. How would you handle this?

A: I would handle this by explicitly listing the required version in the **<dependencyManagement>** section of my main pom.xml. This ensures that all sub-modules and transitive dependencies must use this specific, non-conflicting version. If a library needs the old version, I would use the **<exclusion>** tag to block the old one and let the new one take over.

4. Scenario: You accidentally added a dependency not available in any repository, and the build fails. How would you fix this?

A: I would first run **mvn dependency:tree** to find the exact dependency that is missing and check its coordinates (group ID, artifact ID, version) for typos. If the coordinates are correct, I would verify that the repository containing that dependency is correctly listed in my **settings.xml** or project pom.xml file.

Lifecycle & Build

5. Explain how you would configure a complex Maven build using plugins. Which plugins did you use and why?

A: I configure complex builds by defining goals for specific **plugins** within the <plugins> section of the pom.xml. I use the **maven-compiler-plugin** to specify the required Java version (e.g.,

Java 17). I use the **maven-surefire-plugin** to configure how unit tests should run and the **spring-boot-maven-plugin** to package the final application as an executable JAR file.

6. How can you automate code quality checks (static analysis, style checks) using Maven plugins?

A: I automate code quality checks by integrating dedicated plugins into the **verify** phase of the build lifecycle. I use the **Checkstyle Plugin** to enforce coding style and the **FindBugs/SpotBugs Plugin** or **PMD** to run static analysis, checking for common bugs or code vulnerabilities *before* the package is finalized.

7. How can you use Maven to automatically generate project documentation or reports?

A: I use the **site** lifecycle, which is designed for documentation. I configure the **maven-project-info-reports-plugin** and the **maven-javadoc-plugin** in the pom.xml. Running **mvn site** then automatically generates standardized reports on dependencies, code coverage, and Javadoc documentation in the target/site directory.